Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0117 Programación Paralela y Concurrente

Grupo
03

Profesor:

Josef Ruzicka González

Assignment 0
Enhanced Airport Check-In Simulation

Integrantes:

Jhon Campos Montenegro - C4D674

Gabriel Coto Fernández - C4E540

I Semestre – 2026

Índice

ÍNDICE 2
Explicación de diseño 3
Red de Petro del Diseño de Sincronización 4
Tabla de Orden Global de Locks 5
Escenarios de adquisición múltiple de locks 6
Argumento de Corrección 7
¿Por qué no pueden ocurrir deadlocks? 7
¿Por qué no se sufre de Starvation? 7
Reflexión 8
Desafios Enfrentados 8
DeadLock Entre el Balancer y los Counters 8
Lógica de descanso duplicada 8
Inicialización de los Counters 8
Posibles mejoras 9
Panel en tiempo real 9
Priority Bump al rear en lugar del frente 9
Decisiones de Diseño y Estrategia de Paralelización 10
Modelo de hilos 10
Diseño de la cola: lista enlazada 10
Variables de condición vs semáforos para bloqueo de cola 10
Global Timer y tiempos por pasajero 11
Estrategia de paralelización 11

Explicación de diseño

Esta primer tarea consiste en llevar a cabo la simulación de un **sistema de check-in** en un aeropuerto, con múltiples entidades concurrentes que interactúan a través datos compartidos.

El sistema está compuesto por seis archivos de código fuente distintos. El sistema se caracteriza por implementar hilos POSIX independientes, coordinados mediante mutex, variables de condición y banderas de estado compartidas. A continuación, los seis archivos de código fuente y sus roles en la simulación:

- **main.c**: es el punto de entrada a la simulación, la cual tras procesar los argumentos desde la línea de comandos, inicializa todas las estructuras de datos (las colas), los hilos, espera su respectiva finalización y muestra las estadísticas finales.
- **simulation.c**: define todas las variables globales compartidas (N, M, colas, contadores y estadísticas) por los hilos e implementa create_passengers() que genera la cantidad de pasajeros de acuerdo al valor de N, para posteriormente asignar cada uno a la cola correspondiente (en base a su clase).
- **queue.c**: implementa la estructura de una cola de tipo FIFO, teniendo la composición de una lista enlazada, protegida por un mutex y una variable de condición. Es a partir de ella que se pueden crear las colas para los counters que contienen a los pasajeros.
- **counter.c**: representa a los counters de la simulación. Cada uno de ellos se ejecuta como un hilo independiente; extrayendo hilos de la cola, simulando atenderlos, actualizando estadísticas y tras alcanzada la cantidad de K pasajeros atendidos, entra en descanso.
- **balancer.c**: esta entidad se ejecuta de manera continua durante la simulación, monitoreando el tamaño de las colas y moviendo el exceso de los pasajeros de la cola Economy hacía la cola International. Mientras que, para los pasajeros Business, revisa cuanto tiempo llevan esperando, y en caso de que ese tiempo supere T_max, los mueve al frente de la cola International.
- **supervisor.c**: otra entidad que se ejecuta de forma continua, con el fin de detectar a aquellos counters que deben reabrirse. Para ello los activa después de su tiempo de descanso.

Red de Petro del Diseño de Sincronización

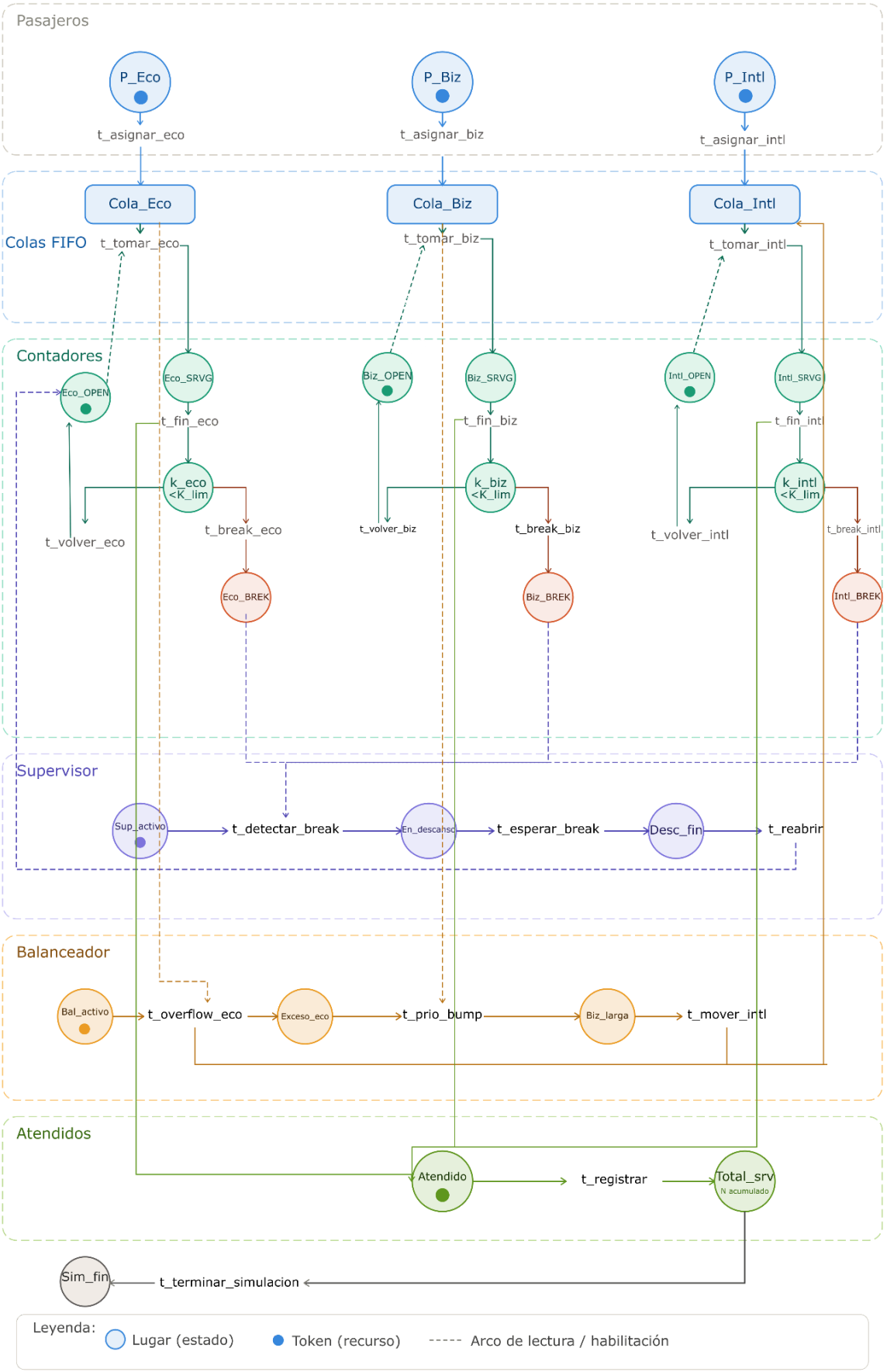

Tabla de Orden Global de Locks

Orden	Mutex	Protege	Hilos que lo usan
1	economyQueue.mutex	front, rear, size de la cola Economy + not_empty condvar	Balancer (lectura de size + dequeue de overflow), counter_thread de tipo Economy (dequeue)
2	businessQueue.mutex	front, rear, size de la cola Business + lógica interna de priority bump	Balancer (recorre nodos para bump), counter_thread de tipo Business (dequeue)
3	internationalQueue.mutex	front, rear, size de la cola International + not_empty condvar	Balancer (enqueue de overflow y pasajeros bumpeados), counter_thread de tipo International (dequeue)
4	remaining_mutex	passengers_remaining, simulation_done	counter_thread (decrementa y activa simulation_done), balancer_thread (condición de salida), supervisor_thread (condición de salida)
5	stats_mutex	total_wait_time, total_service_time, served_economy, served_business, served_international	counter_thread únicamente (acumula stats al terminar cada servicio)

Escenarios de adquisición múltiple de locks

Los siguientes son los únicos hilos que sostienen más de un mutex simultáneamente. En todos los casos, el orden respeta estrictamente la tabla anterior:

Balancer: Overflow de Economy: Adquiere economyQueue.mutex (orden 1) para leer el tamaño. Si hay overflow, llama a dequeue(&economyQueue) que internamente adquiere y libera ese mismo mutex, y luego llama a enqueue(&internationalQueue) que adquiere internationalQueue.mutex (orden 3). Nunca se sostienen ambos al mismo tiempo: se liberan antes de adquirir el siguiente.

Balancer: Priority bump de Business: Adquiere businessQueue.mutex (orden 2) para recorrer los nodos y extraer el pasajero bumpeado. Solo después de liberar ese mutex llama a enqueue(&internationalQueue) que adquiere internationalQueue.mutex (orden 3). Orden respetado: 2 → 3, nunca inverso.

counter_thread: fin de servicio: Adquiere remaining_mutex (orden 4) para decrementar passengers_remaining. Luego, en bloque separado, adquiere stats_mutex (orden 5) para actualizar estadísticas. Nunca se sostienen simultáneamente.

Argumento anti-deadlock: Para que exista un deadlock se requiere un ciclo de espera. El único par de hilos que podría formar un ciclo sería Balancer y counter_thread, ya que ambos acceden a las colas de pasajeros. Sin embargo, el Balancer siempre accede en orden 1 a 3 o 2 a 3, y los counters acceden únicamente a su cola propia (orden 1, 2 o 3 individualmente, nunca combinados). No existe ningún par (A sostiene mutex X esperando Y) + (B sostiene mutex Y esperando X), por lo que el deadlock es imposible por construcción.

Argumento de Corrección

¿Por qué no pueden ocurrir deadlocks?

En primer lugar, un deadlock es cuando dos o más hilos se bloquean mutuamente. Ello pudiendo ser causado por una exclusión mutua, un hold-and-wait (mantiene un recurso mientras espera otro) y una espera circular.

Es entonces que, para evitar dicho problema:

- todo hilo que necesita múltiples mutex los adquiere en un orden fijo: economyQueue, businessQueue, internationalQueue, remaininig_mutex y stats_mutex.
- ningún hilo mantiene un mutex (a) y solicita uno (b) donde (b) < (a), ya que el Balancer unicamente mueve pasajeros de un cola menor (Economy = 0, Business = 1) a la mayor (International = 2). Siendo entonces que, no puede formarse ningun bucle de espera.
- La variable de condición cond_wait(), libera su mutex antes de dormir, por lo que ningún hilo mantiene un mutex mientras está bloqueado. Evitando así, la condición del hold-and-wait.

¿Por qué no se sufre de Starvation?

El problema de Starvation consiste en, resumidas cuentas, cuando un hilo NUNCA obtiene recursos. Por eso, para este trabajo, la implementación del Balancer es fundamental:

- El overflow está basado en un umbral, el cual consiste en que se mueven pasajeros de la cola Economy SOLO cuando la cola supera Q_threshold. Estos son movidos una única vez a la cola International y esperan a ser atendidos.
- El priority bump ocurre solo una vez por pasajero, marcando priority_bumped = true antes de moverlos al inicio de la cola International.
- Los counters que se alimentan de la cola International nunca quedan monopolizados por Business, porque los *bumps* están limitados por el tamaño de la cola Business.

Reflexión

Desafios Enfrentados

DeadLock Entre el Balancer y los Counters

Uno de los desafíos más significativos fue establecer como iban a interactuar el hilo Balancer y los hilos de los counters, ya que cuando ambos necesitaban acceder a dos colas simultáneamente, el balancer adquiría primero internationalQueue.mutex y luego economyQueue.mutex, mientras que un hilo counter podía mantener economyQueue.mutex esperando por internationalQueue.mutex, provocando esto un deadlock.

Para solucionar eso, fue que se implementó el orden indicado en la TABLA DE ORDEN DE LOCKS.

Lógica de descanso duplicada

Inicialmente, el counter_thread manejaba su propio break, esto mediante nanosleep() y luego estableciendo state = OPEN. Sin embargo, supervisor_thread TAMBIÉN hacía lo mismo, es decir, intentando reabrir el mismo counter. Básicamente lo que ocurría era una condición de carrera, donde tanto el counter como el supervisor podían establecer state = OPEN, y el supervisor podía rearbrir un counter que ya se había reabierto a sí mismo.

La solución para ello fue dejarle al supervisor la decisión de reapertura. Así, el counter nada más establece needs_reopen = true y no debe reabrirse a sí mismo.

Inicialización de los Counters

En primera instancia, había un problema con respecto a la inicialización de los counters. Se definía por default tres counters, pero la cantidad de estos tambien se daba por un valor M. Es así que luego en un for i = 0 hasta M para aplicar pthread_create(), se generaban más counters de lo previsto si M > 3, los cuales se llenaban con “información” basura.

La solución para esto fue eliminar la definición por default e implementar una distribución lo más equitativa posible de las tipos de counters a la cantidad de counters, y posteriormente, usar tres bucles para inicializar los counters.

Posibles mejoras

Panel en tiempo real

Añadir que se impriman las longitudes de las colas y los estados de los counters constantemente para visualizar su estado en todo el transcurso de la simulación.

Priority Bump al rear en lugar del frente

El enunciado especifica que un pasajero Business que supera T_max debe ser movido al frente de la cola International. Sin embargo, la función enqueue() implementada en queue.c siempre inserta al rear de la cola. Esto significa que, en la implementación actual, el pasajero bumpeado no recibe prioridad real dentro de la cola International: simplemente se une al final como cualquier otro pasajero.

Para corregir este comportamiento correctamente se requeriría una función enqueue_front() que, manteniendo el mutex de internationalQueue, inserte el nuevo nodo directamente en front y ajuste el puntero rear en caso de que la cola estuviera vacía. Esta función debe adquirir internationalQueue.mutex de manera independiente (sin sostener businessQueue.mutex al mismo tiempo) para respetar el orden global de locks. La ausencia de enqueue_front() no provoca deadlock ni starvation, pero sí debilita la garantía de prioridad que el enunciado exige para pasajeros Business con espera prolongada.

Decisiones de Diseño y Estrategia de Paralelización

Modelo de hilos

Se implementan M hilos para los counters, uno para el balancer y 1 para el supervisor. Esto basado en:

- **Hilos de counters:** cada counter es una unidad de servicio independiente que se bloquea en su cola cuando está vacía. Tener M hilos separados permite servicio paralelo real de pasajeros en múltiples núcleos de CPU..
- **Hilo balancer:** un único balancer evita la complejidad de múltiples balancers compitiendo por mover el mismo pasajero.
- **Hilo supervisor:** de igual forma, un solo supervisor es suficiente. Ya que los counters señalan su necesidad de reapertura mediante una bandera y el supervisor verifica periódicamente.

Diseño de la cola: lista enlazada

Se decidió implementar una lista enlazada ya que N no se conoce en tiempo de compilación, así que esta estructura de datos facilita trabajar con memoria dinámica.

Variables de condición vs semáforos para bloqueo de cola

Se optó por usar variables de condición por sobre semáforos debido a que las primeras son más fáciles de implementar con el mutex (que protege la cola) para mejorar la sincronización y de paso, evitar condiciones de carrera. Así, pthread_cond_wait() libera el mutex y suspende el hilo, garantizando que no se pierdan señales entre la verificación de la cola vacía y el momento de descanso.

Además, las variables de condición permiten usar broadcast(), que facilita la tarea de despertar a todos los counters dormidos de un solo, en lugar de requerir múltiples sem_post(). Además, cuando un productor añade un nuevo pasajero en la cola, utiliza pthread_cond_signal() para despertar a los consumidores bloqueados.

Y otra razón muy importante, es que los semáforos no están tan ligados a un mutex, por lo cual, su implementación aumentaría la complejidad del diseño y el riesgo de errores,

Global Timer y tiempos por pasajero

Para ello se implementan 2 niveles de temporización:

1) un temporizador global: a través del struct timespec con CLOCK_MONOTONIC, se puede medir el tiempo total de la simulación.
2) tiempo por pasajero: se almacena como enteros largos en milisegundos (tv_sec * 1000 + tv_nsec / 1000000). Ya que así se puede dar una precisión suficiente y evita la complejidad de cálculos.

Estrategia de paralelización

La estrategia implementada es el patrón Productor - Consumidor. Esto debido a que se desea implementar un sistema en el que el Balancer, Supervisor y Counters puedan operar de forma concurrente e independiente, interactuando de forma segura mediante las estructuras de datos compartidas.

El objetivo del paralelismo en este trabajo no se limita a solo ejecutar M counters en paralelo, sino que se desea que todas las entidades puedan trabajar de forma simultánea sin bloquearse entre ellas. Así, el modelo se puede detallar de la siguiente manera:

1) Los productores son los encargados de insertar pasajeros en las colas. Tanto al inicio de la simulación como en el transcurso con el Balancer.
2) Los consumidores son los hilos de los counters, que de manera reiterativa extraen pasajeros de las colas para atenderlos.
3) Las colas son el espacio de memoria compartida principal, ya que es allí donde los productores y los consumidores interactúan.

Asimismo, el uso de variables de condición permite que los consumidores se bloqueen de forma adecuada cuando las colas están vacías, evitando así el problema del busy waiting. Mientras que, los productores avisan a los consumidores cuando hay nuevos pasajeros disponibles para que trabajen.

Más específicamente, los hilos Balancer y Supervisor toman en mayor medida una cierta independencia del flujo principal de productor - consumidor. Debido a que el Balancer redistribuye la carga entre colas, mientras que el Supervisor gestiona la disponibilidad de los

counters. Tal que, estos hilos no bloquean directamente a los counters, sino que interactúan mediante la memoria compartida y la sincronización.